

Table des matières

Introduction

Système d'exploitation

Ligne de commande

Processus et fils d'exécution

Utilisateurs et groupes

Fichiers

Gestion de la mémoire

Concurrence et parallélisme

Compilation

Environnement logiciel et virtualisation

Programme

- composition d'un ou plusieurs fichiers source appelés **unités de traduction**

Compilation

- traduction d'un programme écrit dans un langage source en un langage cible
- **C** → **binaire exécutable**, $\text{\LaTeX}$ → PDF, Fortran → assembleur, ...

Étapes (C → binaire exécutable avec gcc) :

1 pré-traitement (C → C)

```
gcc -E fichier-1.c -o fichier-1.i  
gcc -E fichier-2.c -o fichier-2.i
```

2 traduction en assembleur (C → assembleur)

```
gcc -S fichier-1.i -o fichier-1.s  
gcc -S fichier-2.i -o fichier-2.s
```

3 assemblage (assembleur → ELF)

```
gcc -c fichier-1.s -o fichier-1.o  
gcc -c fichier-2.s -o fichier-2.o
```

4 édition de liens (ELF → ELF)

```
gcc fichier-1.o fichier-2.o -o executable
```

Pré-traitement

 $C \rightarrow C$

- remplacement **récuratif** des directives `#include` par le contenu des fichiers inclus

Localisation de fichiers

- emplacements habituels `/usr/include` et `/usr/local/include`
- dossiers spécifiés par la variable d'environnement `C_INCLUDE_PATH` ou `CPP_INCLUDE_PATH`
 - par exemple `C_INCLUDE_PATH=/home/user/more/includes gcc -o my my.c`
- emplacements spécifiés par l'utilisateur à l'aide de l'option `-I` du compilateur
 - par exemple `gcc -I/home/user/more/includes -o my my.c`

Pré-traitement

 $C \rightarrow C$

- évaluation et remplacement des macros par leurs définitions

- macros pré-définies par le compilateur

Macro	Contenu
<code>__FILE__</code>	fichier courant
<code>__FUNC__</code>	fonction courante
<code>__LINE__</code>	ligne actuelle
<code>__DATE__</code>	date actuelle
<code>__TIME__</code>	heure actuelle

- macros définies avec des directives `#define`
- macros définies avec l'option `-D` du compilateur
- évaluation des directives conditionnelles `#if`, `#ifdef` et `#ifndef`

Compilation en fonction de l'environnement

- combinaison des directives conditionnelles et des macros pré-définies en fonction de la configuration locale par le compilateur, le système, les bibliothèques, ...
 - `__WIN32` – uniquement sur Windows
 - `__APPLE` – uniquement sur MacOS
 - `__linux__` – uniquement sur Linux
 - `__OPENMP` – uniquement en présence d'une installation fonctionnelle d'OpenMP
 - etc.

Pré-traitement

Exemple : division personnalisée

Avant le pré-traitement :

■ fichier « div.c »

```
#include <stdio.h>
#include "div.h"
void div(float a, float b) {
    if(b > 0)
        printf(
            "%s: %.2f / %.2f = %.2f\n",
            PROGRAM, a, b, a / b
        );
    else
        ERROR("division by zero");
}
```

■ fichier « div.h »

```
// Pour éviter des définitions répétées
// si 'div.h' est inclus plusieurs fois
#ifndef __DIV_H
#define __DIV_H
#ifdef __linux__
#define PROGRAM "Division on Linux"
#else
#define PROGRAM "Division on other system"
#endif
#define ERROR(msg) printf("%s\n", msg)
void div(float a, float b);
#endif
```

Pré-traitement

Exemple : division personnalisée

Après le pré-traitement :

- fichier « div.c » sur Linux

```
// contenu de `stdio.h`  
void div(float a, float b);  
void div(float a, float b) {  
    if(b > 0)  
        printf(  
            "%s: %.2f / %.2f = %.2f\n",  
            "Division on Linux",  
            a, b, a / b  
        );  
    else  
        printf("%s\n", "division by zero");  
}
```

- fichier « div.c » sur un autre système

```
// contenu de `stdio.h`  
void div(float a, float b);  
void div(float a, float b) {  
    if(b > 0)  
        printf(  
            "%s: %.2f / %.2f = %.2f\n",  
            "Division on other system",  
            a, b, a / b  
        );  
    else  
        printf("%s\n", "division by zero");  
}
```

Pré-traitement

Exemple : « Bonjour » conditionnel avec division personnalisée

Avant le pré-traitement :

■ fichier « hello.c »

```
#include <stdio.h>
#include "div.h"
#define GET_VALUE(x) (x+1)
int main() {
    #ifdef HELLO
    #if HELLO == 1
        printf("Hello from %s\n", __FILE__);
    #elif HELLO == 2
        printf("%d\n", GET_VALUE(5));
        div(1.0, 2.0);
    #endif
    #endif
    return 0;
}
```

Pré-traitement

Exemple : « Bonjour » conditionnel avec division personnalisée

Après le pré-traitement :

- fichier « hello.c »

```
// contenu de `stdio.h`  
void div(float a, float b);  
int main() {  
    return 0;  
}
```

- fichier « hello.c » compilé avec -DHELLO=1

```
// contenu de `stdio.h`  
void div(float a, float b);  
int main() {  
    printf("Hello from %s\n", "hello.c");  
    return 0;  
}
```

- fichier « hello.c » compilé avec -DHELLO=2

```
// contenu de `stdio.h`  
void div(float a, float b);  
int main() {  
    printf("%d\n", (5+1));  
    div(1.0, 2.0);  
    return 0;  
}
```

Traduction en assembleur

C → assembleur

- traduction du code source C en assembleur pour l'architecture cible (x86_64, arm, ...)
- optimisation du code source écrit par le programmeur
 - plusieurs passes (élimination d'invariants de boucle, vectorisation, ...)
 - ciblée (ex. `-ftree-vectorize` uniquement pour la vectorisation)
 - par niveaux (options `-O0`, `-O1`, `-O2` et `-O3`; `-O2` par défaut avec `gcc`)

Sous-étapes principales :

- 1 analyse lexicale
 - flux de caractères → éléments de langage
- 2 analyse syntaxique
 - éléments de langage → représentation intermédiaire de haut niveau-arbre de syntaxe abstraite (AST)
- 3 analyse sémantique
- 4 optimisations sur AST
- 5 génération de code
 - AST → représentation intermédiaire de bas niveau-code à trois adresses (LLVM IR, ...)
- 6 optimisations sur code à trois adresses
- 7 production de l'assembleur
 - code à trois adresses → assembleur

Traduction en assembleur

Sous-étapes principales

1 analyse lexicale

- identification des éléments du langage
- ex. `int var = "hello";` → type, espace, identifiant, espace, chaîne de caractères, point-virgule
- ex. `int 2var;` → type, espace, '2', 'v', 'a', 'r', point-virgule

2 analyse syntaxique

- vérification de la conformité avec la grammaire
- traduction en une représentation intermédiaire de haut-niveau—arbre de syntaxe abstraite (AST)
- ex. type, espace, identifiant, espace, chaîne de caractères, point-virgule → succès, déclaration de variable avec initialisation
- ex. type, espace, '2', 'v', 'a', 'r', point-virgule → erreur, identifiant invalide

Traduction en assembleur

Sous-étapes principales

- 3 analyse sémantique
 - vérification du sens du programme
 - ex. déclaration d'une variable de type `int` initialisée avec une chaîne de caractères → erreur
- 4 optimisations sur AST
- 5 génération de code à trois adresses (instruction, 2 opérandes + 1 résultat)
 - traduction en une représentation de bas niveau proche de l'assembleur mais indépendante de l'architecture cible
- 6 optimisations sur code à trois adresses
- 7 génération de l'assembleur pour l'architecture cible

Traduction en assembleur

Exemple : « Bonjour » simple

■ fichier « simple-hello.c »

```
#include <stdio.h>
int main() {
    printf("Hello world!\n");
    return 0;
}
```

■ fichier « simple-hello.s »

■ tronqué, assembleur x86_64

```
.file          "simple-hello.c"
.text
.section       .rodata
.LC0:
.string       "Hello world!"
.text
.globl        main
.type        main, @function
...
```

```
...
main:
.LFB0:
.cfi_startproc
endbr64
pushq        %rbp
.cfi_def_cfa_offset 16
.cfi_offset  6, -16
movq         %rsp, %rbp
.cfi_def_cfa_register 6
leaq         .LC0(%rip), %rax
movq         %rax, %rdi
call         puts@PLT
movl         $0, %eax
popq         %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
...
```

Traduction en assembleur

Exemple : optimisation de code (exprimé en C pour plus de lisibilité)

Avant optimisation :

■ fichier « opt.c »

```
#include <stdio.h>
int main() {
    for(int i = 0; i < 10; i++) {
        int base = 1;
        printf("Iteration no. %d\n", base + i);
    }

    return 0;
}
```

Après élimination d'invariants de boucle :

■ fichier « opt.c » optimisé

```
#include <stdio.h>
int main() {
    int base = 1;
    for(int i = 0; i < 10; i++) {
        printf("Iteration no. %d\n", base + i);
    }

    return 0;
}
```

Motivation

Il est inutile de répéter la définition de base qui a la même valeur à chaque tour de la boucle sur i.

Assemblage

assembleur → ELF

- traduction du code source de l'assembleur en langage machine

Assembleur

- langage de programmation de bas niveau
- spécifique à chaque architecture
- x86_64 (Intel, AMD, ...), arm (Apple M, ...), RISC-V, MIPS, ...

Langage machine

- langage binaire illisible pour un humain
- spécifique à chaque architecture
- ELF sur Linux

Édition de liens

ELF → ELF

- rassemblement d'une ou plusieurs unités de traduction en langage machine
- création d'un exécutable (ELF sur Linux)

Exemple : compilation de « div.c » et de « hello.c » en un binaire exécutable nommé « hello »

■ arborescence de fichiers

```
.  
|-- div  
|   |-- div.h  
|   |-- div.c  
|-- hello.c
```

1 pré-traitement, traduction en assembleur et assemblage

- unité de traduction « div.c »

```
gcc -o div.o -c div/div.c
```

- unité de traduction « hello.c »

```
gcc -o hello.o -I./div -c hello.c
```

2 édition de liens et production de « hello »

```
gcc -o hello div.o hello.o
```

Bibliothèques

Généralités

- **composants logiciels** implémentant un ensemble de fonctionnalités **réutilisables** lors de la conception de nouvelles applications
 - bibliothèques multi-fonctionnelles (libc, ...)
 - bibliothèques mathématiques (BLAS, FFTW3, MUMPS, ...)
 - bibliothèques de chiffrement (SSL, ...)
 - bibliothèques de programmation parallèle (pthreads, OpenMPI, ...)
 - etc.

Distribution

- implémentation des fonctionnalités de la bibliothèque
- interface de programmation (API) donnant accès à ces fonctionnalités

Bibliothèques

Cas des bibliothèques en C sur Linux

Distribution

- implémentation des fonctionnalités de la bibliothèque
 - fichiers binaires pré-compilés (ex. `libtoto.a` ou `libtoto.so`) pour une architecture donnée (`x86_64`, `arm`, ...)
 - installation habituelle dans `/usr/lib` ou `/usr/local/lib`
- interface de programmation (API) donnant accès à ces fonctionnalités
 - fichiers en-tête (ex. `toto.h`) exposant les déclarations des fonctions et des variables utiles
 - installation habituelle dans `/usr/include` ou `/usr/local/include`

Bibliothèques

Cas des bibliothèques en C sur Linux

Compilation d'une application « app » faisant appel à une bibliothèque « toto » :

- bibliothèque installée sous /usr, /usr/local ou dans un des emplacements indiqués par les variables d'environnement C_INCLUDE_PATH (fichiers en-tête) et LD_LIBRARY_PATH ou LIBRARY_PATH (fichiers binaires)

```
gcc -o app app.c -ltoto
```

- bibliothèque installée dans un autre dossier (ex. /home/user/toto avec les fichiers en-tête dans include et les fichiers binaires dans lib)

```
gcc -o app app.c -I/home/user/toto/include -L/home/user/toto/lib -ltoto
```

Bibliothèques

Bibliothèques statiques

- liées et injectées dans le binaire de l'application utilisatrice à la compilation

Distribution

- fichiers binaires *.o de la bibliothèque regroupés dans une archive *.a (ex. libtoto.a) créée avec la commande ar (ex. ar rcs libtoto.a toto1.o toto2.o)

Compilation d'une application « app » faisant appel à une bibliothèque **statique** « toto » :

- bibliothèque installée sous /usr, /usr/local ou dans un des emplacements indiqués par les variables d'environnement C_INCLUDE_PATH (fichiers en-tête) et LIBRARY_PATH (fichiers binaires)

```
gcc -o app app.c -l:libtoto.a
```

- bibliothèque installée dans un autre dossier (ex. /home/user/toto avec les fichiers en-tête dans include et les fichiers binaires dans lib)

```
gcc -o app app.c -I/home/user/toto/include -L/home/user/toto/lib -l:libtoto.a
```

Bibliothèques

Bibliothèques partagées

- référencées à la compilation, mais injectées à l'exécution de l'application utilisatrice
- localisation (par ordre de priorité) des fichiers binaires des bibliothèques à l'exécution
 - 1 aux emplacements indiqués dans la section `rpath` de l'exécutable de l'application utilisatrice (modifiable avec l'option `-Wl,-rpath` du compilateur)
 - 2 aux emplacements indiqués par la variable d'environnement `LD_LIBRARY_PATH`
 - 3 aux emplacements indiqués dans la section `runpath` de l'exécutable de l'application utilisatrice
 - 4 aux emplacements habituels dans `/usr/lib` et `/usr/local/lib`

Distribution

- fichiers binaires `*.o` de la bibliothèque regroupés en un fichier `*.so` (pour *shared object*, ex. `libtoto.so`) compilé avec la commande `gcc` (ex. `gcc -o libtoto.so toto1.o toto2.o -shared -fPIC`)
 - `-fPIC` pour *Position-Independent Code*—code de la bibliothèque sera exécutable indépendamment de son emplacement dans la mémoire virtuelle du processus de l'application utilisatrice

Bibliothèques

Bibliothèques partagées

Compilation d'une application « app » faisant appel à une bibliothèque **partagée** « toto » :

- bibliothèque installée sous `/usr`, `/usr/local` ou dans un des emplacements listés dans les variables `C_INCLUDE_PATH` (fichiers en-tête) et `LD_LIBRARY_PATH` (fichiers binaires)

```
gcc -o app app.c -ltoto
```

- bibliothèque installée dans un autre dossier (ex. `/home/user/toto` avec les fichiers en-tête dans `include` et les fichiers binaires dans `lib`)

```
gcc -o app app.c -I/home/user/toto/include -L/home/user/toto/lib  
-Wl,-rpath,/home/user/toto/lib -ltoto
```

- à l'exécution, « app » cherchera les fichiers binaires de la bibliothèque d'abord dans `/home/user/toto/lib`

Bibliothèques

Récapitulatif

Bibliothèques statiques

- fichiers binaires injectés dans l'exécutable de l'application utilisatrice à la compilation
- application utilisatrice indépendante de l'installation de la bibliothèque à l'exécution
- exécutable de l'application utilisatrice plus grand

Bibliothèques partagées

- fichiers binaires référencés depuis l'exécutable de l'application utilisatrice à la compilation et chargés seulement à l'exécution
- exécution de l'application utilisatrice nécessite une installation fonctionnelle de la bibliothèque sur la machine cible
- exécutable de l'application utilisatrice plus léger
- possibilité de changer d'implémentation de bibliothèque sans recompiler l'application utilisatrice

Bibliothèques

Exemple : suite de Fibonacci

- deux implémentations (`fibosimple` et `fiboclever`) d'une bibliothèque qui calcule la suite de Fibonacci
- une application (`fibotest`) pour tester ces implémentations
- instructions dans le fichier `README.md`
- exemple à télécharger depuis <https://felsoci.sk/aise/fibonacci.zip>

Bogue

- terminaison pré-maturée du programme
- production d'un résultat inattendu
- inter-blocage

Déboguage

- suivre l'exécution du programme (flot de données et d'instructions) afin de localiser le problème
- analyser le résultat produit

Déboguage

Quelques méthodes pour les logiciels écrits en C

- injection d'appels à une fonction d'affichage telle que `printf`
 - méthode naïve, parfois suffisante
- utilisation d'un débogueur
 - générique (GDB, ...)
 - mémoire (valgrind, ...)
 - précision numérique (CADNA², ...)
- traçage d'appels système (strace)
- suivi de la séquence de chargement de bibliothèques partagées
 - exécution du programme en positionnant la variable d'environnement `LD_DEBUG` à `libs`
 - ex. `LD_DEBUG=libs ./mon-programme`

2. <https://cadna.lip6.fr/>